

Metaheuristic Optimization for Multi-AGV Path Planning in Search and Rescue Missions

Arwa H. Ghoneim^{1*}, Ahmed M. Elsoda¹, Heba A. Abdelmawla¹, Khaled Elbadawy¹

Linah Tamer¹, Lobna Tarek¹, Jessica Magdy^{1,2}, Omar M. Shehata^{1,2}

¹*Mechatronics Engineering Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt*

²*Multi-Robot Systems (MRS) Research Group, Cairo, Egypt*

{arwa.h.ghoneim, ahmedelsoda287, heba.ashraf.abdelmawla, elbadawy2002khaled, linahatamer, lobnaterek48}@gmail.com, {jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—This paper addresses optimizing multi-Autonomous Ground Vehicle (AGV) coordination in autonomous ground search and rescue missions, focusing on enhancing exploration efficiency through collaborative path planning. By modeling the environment as a 2D grid and leveraging optimization algorithms, the aim is to maximize area coverage while minimizing exploration time. The optimization problem is formulated based on travel distance and the number of explored grid cells. Simulated annealing (SA), Genetic algorithm (GA), Particle Swarm Algorithm (PSO) and Artificial Protozoa Optimizer (APO) are applied to generate optimized paths for multiple AGVs, ensuring efficient task distribution. The performance of each algorithm is evaluated based on the overall objective function and computational time. The results demonstrate the framework's capability to improve exploration efficiency and path allocation, making it a suitable approach for AGV coordination in large-scale environments.

Index Terms—multi-AGV, metaheuristic optimization, path planning, simulated annealing, genetic algorithm, particle swarm optimization, artificial protozoa optimizer.

I. INTRODUCTION

In recent years, Autonomous Ground Vehicles (AGVs) have gained significant attention for their potential in a wide range of applications, particularly in disaster response scenarios where human intervention poses significant risks. In environments such as those affected by earthquakes, fires, or other catastrophic events [1]. Rapid and efficient search and rescue efforts become critical to saving lives. In such hazardous environments, human intervention is often too dangerous or impractical, making AGVs an invaluable asset. These autonomous vehicles can navigate through collapsed structures, hazardous terrains, and unpredictable conditions to locate survivors, deliver essential supplies, and provide real-time data to rescue teams. Additionally, AGVs are built to operate in environments that may contain toxic gases, unstable structures, or other conditions that pose serious risks to human health. Their ability to operate tirelessly in high-risk areas not only enhances rescue efficiency but also reduces the risk faced by human responders. However, for AGVs to be effective in such life-saving missions, they must be able to explore large areas in the shortest possible time. This requires advanced optimization techniques to generate precise, collision-free paths. To address these challenges, several optimization algorithms were explored, including Simulated Annealing (SA) [2], [3], Genetic Algorithms (GA) [4], [5], Particle Swarm Optimization (PSO)

[6], [7], and Artificial Protozoa Optimization (APO) [8]. Each of these algorithms offers unique advantages in optimizing both the path and the exploration time of AGVs, ensuring that they can operate effectively in disaster-stricken areas. The primary aim of this paper is to optimize the path of multiple AGVs to maximize the exploration of the target area while minimizing the overall exploration time, subject to various constraints. Key constraints include ensuring that AGVs explore as much of the area as possible without overlapping with each other's paths, and maintaining operational efficiency.

There is a study for hybrid optimization approach for AGV path planning (KGWO) by integrating Grey Wolf Optimization (GWO) with a Kalman filter, partially matched crossover (PMX) mutation, and roulette wheel selection to enhance path accuracy, smoothness, and efficiency in complex environments. The optimization process begins with GWO, which mimics the hunting behavior of grey wolves, where the alpha, beta, and delta wolves guide the search for the optimal path by balancing global exploration and local exploitation. A fitness function evaluates solutions based on path length, smoothness, and obstacle avoidance, ensuring that AGVs navigate efficiently. To further refine the best solution, a Kalman filter is applied every ten iterations to adjust and stabilize the selected path by using prediction and correction steps, keeping the AGV on a smooth and steady route. The study formulates a multi-objective fitness function that combines path length, smoothness, and obstacle avoidance into a weighted sum, allowing for optimized AGV trajectory planning. The results showed that KGWO significantly reduced path length by 13.6% compared to traditional GWO while also improving convergence speed, obstacle avoidance accuracy, and overall stability [9].

Haba et al. (2022) explored the use of reverse annealing to optimize travel time in multi-AGV routing, aiming to reduce detours and improve efficiency in automated logistics systems. Their study leverages quantum annealing techniques, comparing reverse annealing with both forward annealing and the classical solver Gurobi. Simulations in a virtual plant showed that reverse annealing not only outperforms forward annealing but also delivers up to a tenfold speed improvement over Gurobi. The findings highlight the potential of quantum computing for real-time multi-AGV coordination, presenting a

promising alternative to traditional optimization methods. This research contributes to the work on AGV path optimization by introducing advanced computational approaches that enhance operational efficiency [10].

QiuYun et al. (2021) present an improved particle swarm optimization (IPSO) algorithm for AGV path planning, focusing on minimizing transportation time while maintaining computational efficiency. Their approach enhances the standard PSO algorithm with an integer-coding method, crossover operations, and a mutation mechanism to prevent premature convergence. The study evaluates IPSO against GA, ACO, and standard PSO, demonstrating its superiority in achieving shorter path times. These enhancements align with the objectives of search and rescue AGVs, where optimizing exploration while minimizing path time is crucial. The IPSO framework provides valuable insights into improving metaheuristic-based AGV navigation in time-sensitive applications [11].

This paper first reviews existing literature and related work on AGV path and time optimization, highlighting previous approaches and their limitations. Then, the methodology is presented, detailing the problem formulation and the application of each optimization algorithm, including parameter selection and implementation details. This is followed by a discussion of the experimental results, where the performance of the four algorithms is evaluated and compared. The paper concludes by summarizing key findings and insights from the analysis.

II. METHODOLOGY

This paper aims to optimize the exploration paths for multi-AGVs, achieving autonomous synchronization for the AGVs to explore the maximum area possible in minimum time.

A. Problem formulation

To accurately define this problem, some assumptions were made, the size of each AGV is smaller than a grid cell, the view of each AGV is equivalent to the size of one grid cell, each AGV moves at a constant velocity v , and is limited to pass by a fixed number of points. Each AGV has a unique start point while having the same endpoint as the rest of the fleet. A path p_k for AGV_k is defined as a sequence of points starting from S_k and ending at E with n as the number of intermediate points in the path:

$$p_k = [S_k = p_{k,0}, p_{k,1}, p_{k,2}, \dots, p_{k,n}, p_{k,n+1} = E], \quad (1)$$

The solution array is represented in the form of:

$$\begin{bmatrix} x_{1,S} & x_{1,1} & x_{1,2} & \cdots & x_{1,n} & x_{1,E} \\ y_{1,S} & y_{1,1} & y_{1,2} & \cdots & y_{1,n} & y_{1,E} \\ x_{2,S} & x_{2,1} & x_{2,2} & \cdots & x_{2,n} & x_{2,E} \\ y_{2,S} & y_{2,1} & y_{2,2} & \cdots & y_{2,n} & y_{2,E} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{K,S} & x_{K,1} & x_{K,2} & \cdots & x_{K,n} & x_{K,E} \\ y_{K,S} & y_{K,1} & y_{K,2} & \cdots & y_{K,n} & y_{K,E} \end{bmatrix} \quad (2)$$

where the matrix contains K AGVs, with the odd rows representing the x-coordinate and even rows representing the

y-coordinates. For each AGV, the first column corresponds to the start point S , the intermediate points are labeled $1, 2, \dots, n$ and the last column corresponds to the endpoint E . An example of the solution is visually shown in Fig. 1.

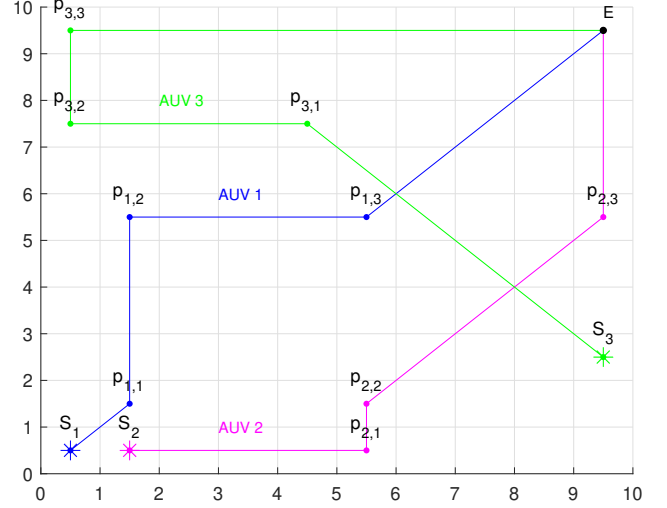


Fig. 1. 10x10 grid with the path of 3 AGVs with different start points and reaching the same endpoint ($K = 3, n = 3$)

The decision variables for this optimization problem are the starting point S_k of each AGV_k and the n intermediate points along the path p_k , specifically the points $p_{k,i}$ for $i = 1, 2, \dots, n$. The distance between two consecutive points on the path is calculated using euclidean distance formula.

$$distance(p_{k,i}, p_{k,i+1}) = \sqrt{(x_{k,i} - x_{k,i+1})^2 + (y_{k,i} - y_{k,i+1})^2}. \quad (3)$$

The total path length for AGV_k is:

$$Length(p_k) = \sum_{i=0}^n distance(p_{k,i}, p_{k,i+1}), \quad (4)$$

and the exploration time for each AGV is given by:

$$time_k = \frac{Length(p_k)}{v}, \quad (5)$$

where v is the constant velocity.

For the time objective function, the exploration time is defined as the maximum time taken by any individual AGV, ensuring all AGVs complete their exploration within this duration. The overall function for exploration time is therefore:

$$T = \max_{k=1}^K time_k, \quad (6)$$

where K is the total number of AGVs.

The exploration area is determined by counting all unique cells explored by any AGV. A cell c in the grid is considered explored by AGV_k if the shortest distance from the cell to any segment on the AGV's path is less than a specified threshold ϵ . Let $distance(c, p_{k,i}, p_{k,i+1})$ denote the shortest distance from the cell c to the segment between points $p_{k,i}$ and $p_{k,i+1}$ on the path of AGV_k . The cell c is explored if:

$$distance(c, p_{k,i}, p_{k,i+1}) < \epsilon \quad (7)$$

for any segment $(p_{k,i}, p_{k,i+1})$ on the path of AGV_k . In this case, the threshold ϵ is set to 1, meaning a cell is considered explored if it lies within one cell's distance from any path segment. This captures adjacent cells in all directions—including diagonal proximity—ensuring effective coverage representation.

The total exploration area A can then be calculated as the sum of all cells c that satisfy this condition:

$$A = \sum_{i=1}^M \sum_{j=1}^O \text{explored}_{i,j}, \quad (8)$$

where $\text{explored}_{i,j} = 1$ if the cell (i, j) is explored (i.e., its distance to any path segment is less than ϵ), and $\text{explored}_{i,j} = 0$ otherwise. Here, M and O represent the grid dimensions in the x (lateral) and y (longitudinal) directions, respectively. The total objective function is developed to be:

$$P = \max \left(w_e \cdot A + \frac{w_t}{T + 0.001} \right), \quad (9)$$

where w_e and w_t are the weights for exploration area and time, respectively. Given that the typical magnitudes of the exploration area exceed those of the exploration time by a factor of approximately 100,000, the weights are assigned as $w_e = 1$ and $w_t = 100,000$ to appropriately balance the contributions of each parameter.

Furthermore, the path points generated were constrained within the grid,

$$x_{k,i} \in \{1, \dots, M\}, \quad y_{k,i} \in \{1, \dots, O\}. \quad (10)$$

Additionally, a maximum exploration time was set for all AGVs to account for the AGV's battery life cycle,

$$T \leq T_{max}. \quad (11)$$

To ensure adequate dispersion across the grid, a minimum distance constraint is imposed between each pair of consecutive points in the path,

$$\text{distance}(p_{k,i}, p_{k,i+1}) \geq d_{min}. \quad (12)$$

Lastly, AGVs are treated as obstacles to each other and must avoid collisions. To enforce this, the constraint $d_{k,l} \neq d_{l,l} \quad \forall k \neq l$ is imposed such that if their paths intersect, the cumulative distances traveled by the AGVs up to the intersection point must differ. This approach is feasible due to the constant and identical velocity of all AGVs.

The cumulative distance for AGV k at the intersection point $I_{k,l}$ is:

$$d_{k,l} = d_{k,i} + \text{distance}(p_{k,i}, I_{k,l}), \quad (13)$$

where $d_{k,i}$ is the cumulative distance traveled by AGV k up to point $p_{k,i}$, and $\text{distance}(p_{k,i}, I_{k,l})$ is the Euclidean distance between $p_{k,i}$ and the intersection point $I_{k,l}$. Similarly, the cumulative distance for AGV l at the intersection point $I_{k,l}$ is:

$$d_{l,l} = d_{l,j} + \text{distance}(p_{l,j}, I_{k,l}), \quad (14)$$

The parameters for the base case used across all algorithms in this optimization scenario are detailed in table I.

TABLE I
BASE CASE PARAMETERS

Parameters	
Number of AGVs (K)	5
Number of intermediate path points (n)	5
AGV battery life (T_{max})	50 hours
Minimum distance between consecutive points (d_{min})	10 meters
Number of iterations	300 iterations
Population size for population-based algorithm	10

B. Simulated annealing

SA is a powerful metaheuristic optimization technique inspired by the annealing process in metallurgy, where materials are heated and gradually cooled to achieve a stable crystalline structure [2]. This method is particularly effective for solving complex optimization problems with large search spaces and numerous local optima. In this study, SA applied to optimize the mission planning of AGVs to balance exploration coverage and mission time. Adding to the parameters mentioned in the problem formulation, the base case study for this algorithm has a scaler of 0.3. The latter is used to scale the random noise added to the previous solution to create a new one.

A linear cooling schedule was employed to progressively reduce the acceptance probability of suboptimal solutions, allowing the algorithm to escape local minima and converge near the global optimum. For iteration i : $T_i = T_0 - \beta * i$, where T_0 is the initial temperature and T_i is the temperature for the current iteration, β is a parameter determining the rate of cooling. The aforementioned probability is calculated as follows:

$$\text{probability} = \exp \frac{-(P_{n+1} - P_n)}{T_i}, \quad (15)$$

where (P_{n+1}) is the current worse solution, (P_n) the previous solution, and T is the temperature for the current iteration. This probability is then compared to a random number r to decide the acceptance.

C. Genetic algorithm

GA is an optimization technique inspired by the process of natural selection and evolution. It simulates reproduction, mutation, and survival of the fittest (elite) to find optimal solutions [4]. The algorithm begins with a population of random solutions, which are evaluated based on their fitness. These solutions undergo crossover to create offspring, with occasional mutations to enhance exploration. Over successive generations, the population evolves as the fittest individuals pass on their genetic material, gradually improving solution optimality. GA is particularly effective for solving complex optimization problems with large, multi-dimensional search spaces, where traditional methods may struggle. In this study, GA is applied to optimize AGV mission planning. The base case scenario includes 10 individuals. To balance exploration and exploitation, the algorithm uses 10% elitism, 60% crossover, and 30% mutation. Crossover operations leverage an alpha value of 0.4, while mutations are guided by a

noise scale of 0.4. These parameters were carefully chosen to ensure the algorithm navigates the multi-dimensional search space efficiently, producing solutions that optimize exploration coverage and resource utilization for AGV missions.

D. Particle swarm optimization

PSO, introduced by Kennedy and Eberhart in 1995, is a population-based metaheuristic inspired by swarm behaviors like bird flocks or fish schools [6]. Known for its simplicity and efficiency, PSO iteratively updates particles' positions in the search space based on individual and collective experiences, balancing exploration and exploitation to find optimal solutions. It has broad applications in engineering, machine learning, and scheduling.

For the base study implementation of PSO for AGV mission planning, parameters included an inertia weight of 0.792 for exploration-exploitation balance and acceleration coefficients of 1.5 for personal and global best convergence.

E. Artificial protozoa optimizer

APO draws inspiration from the survival mechanisms of protozoa, single-celled organisms, simulating behaviors such as foraging, dormancy, and reproduction to execute optimization tasks in metaheuristic algorithms [8]. In the context of this study, APO is used to optimize the path planning of multiple AGVs in search and rescue missions. Each AGV is represented as an artificial protozoa, and the algorithm simulates their collective behavior to efficiently explore the search space. Based on the proportion factor ($p_f = 0.1$ for the base case), a portion of the protozoa population enters dormancy, where the new solution is replaced by a random solution, or undergoes reproduction, where the new solution is derived from the old solution with added noise. The remaining protozoa forage for better solutions using two modes: heterotroph and autotroph. In this case, the foraging behavior considers the influence of neighbor pairs to adjust the position of the protozoa, with the base case using only one neighbor pair ($n_p = 1$).

III. RESULTS AND DISCUSSION

This section illustrates the results of the base case for the SA, GA, PSO, and APO algorithms in addition to comparing different cases. To evaluate the performance of the different algorithms, the convergence rate was observed, along with best cost function, the average cost function, the standard deviation to assess the repeatability of each algorithm, and the computational time among 15 runs for the same base case in each algorithm.

In the graphs, the legend uses black symbols for clarity, but each symbol is represented in five distinct colors (excluding black) corresponding to the five AGVs. Specifically, the solid diamond (◆) represents the current best start point for each AGV, while the empty diamond (◇) denotes the start point in the current iteration for each AGV. The solid line (—) indicates the best path found for each AGV, and the dashed line (- -) represents the path of each AGV in the current iteration. This

approach avoids redundancy in the legend while ensuring the distinction between AGVs is maintained through color coding.

The results of the base case of the SA algorithm are shown in Fig. 2.

The scattering of the objective function's values across itera-

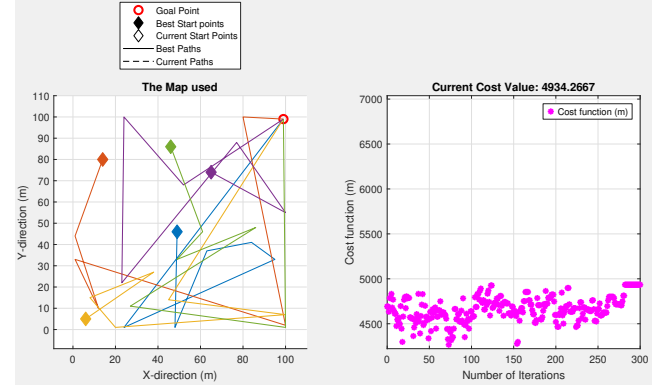


Fig. 2. SA algorithm results for the base case

tions, as observed in the second graph of Fig. 2, results from the inherent structure of the SA algorithm, where the latter occasionally accepts worse solutions based on a probability factor. The results demonstrate that the SA algorithm effectively maximizes exploration while minimizing travel time. As the first graph shows, the paths found by the algorithm for the AGVs were able to explore a large area of the grid. In the case studies, doubling path points (from $n = 5$ to $n = 10$) or increasing AGVs ($k = 5$ to $k = 10$) raised considerably the cost function by nearly 24% due to greater exploration potential. While adding path points improved results, it often violated minimum distance or maximum time constraints, requiring higher price like larger battery sizes (therefore choosing bigger T_{max}). Similarly, increasing AGVs enhanced exploration but led to several overlaps between AGVs and longer computation times (increased by nearly 47%). Switching to geometric cooling ($T_i = T_0 * \alpha^i$ where $\alpha = 0.8$) produced curves resembling those observed if the acceptance probability was ignored. These curves exhibited rapid stagnation, with increased exploitation and reduced exploration. Parameter adjustments such as iterations (from 300 to 1000 iterations) and scaler value (from 0.3 in base case to 0.9) often yielded negligible or worse results. Despite reducing the risk of local minima, the inclusion of probability caused oscillations, particularly with large scalar values, often leading to worse final cost functions than when the probability is disregarded.

For the GA algorithm, the results are shown in Fig. 3. In the figure, the GA cost function curve on the right improves with each generation reaching an approximate value of 5000. This result shows that GA is more suitable than SA for this optimization problem. As with SA, the best cost function values were achieved by doubling path points (22% rise) and AGVs number (38% increase), but at the expense of higher battery demands or excessive computation time (66% longer than the

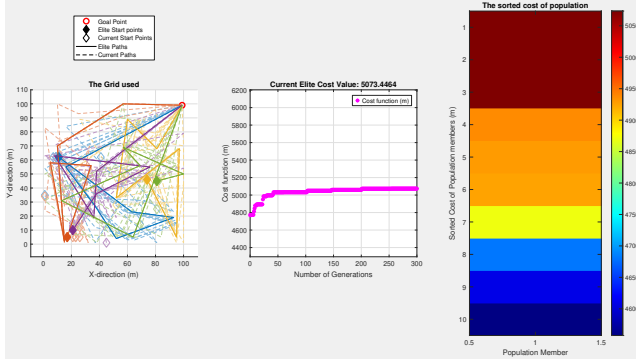


Fig. 3. GA algorithm results for the base case

base case for path points and 3.27 times longer for AGVs number). Increasing generations to 500 slightly improved the cost function, but increased the computation time by 69%. Adjusting elite, mutation, and crossover ratios had minimal impact, with mutation performing the best. Decreasing alpha to 0.1 slightly improved the cost function with a modest increase in computation time, outperforming the default case. Doubling population size and increasing noise scale from 0.4 to 0.9 offered minor cost improvements but nearly doubled computation time, making these strategies inefficient.

The results of the PSO algorithm for the base case are presented in Fig. 4. The analysis of the optimization results using

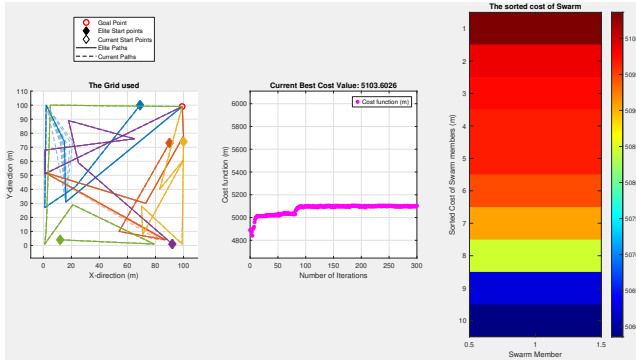


Fig. 4. PSO algorithm results for the base case

PSO across various cases shows notable trade-offs between cost function values, convergence speeds, and computational complexity. The default case achieved a moderate cost function with the shortest computation time among all cases. Doubling the path points, and consequently maximum time constraint (from $T_{max} = 50$ to 100) raised the cost by 14% with a corresponding 14% increase in computation time. Doubling the number of AGVs resulted in the highest cost function with a 34.43% increase but it took three times longer to finish all iterations. Furthermore, doubling the iterations slightly reduced the cost but lengthened the processing by 2.5 times. Adjusting the inertia weight factor to 0.5 produced results similar to the default, while increasing it led to failure in finding feasible solutions. Decreasing acceleration coefficients

relative to global and personal bests had mixed impacts, with the global best achieving almost the same cost function as the default but in 25% more computation time, while the personal best achieved the lowest cost (6% less than the default) and required 35% more processing time. Finally, doubling the swarm size resulted in the fastest convergence (less than 15 iterations) with a comparable cost function but twice the computation time.

The APO results are shown in Fig. 5. In the case of

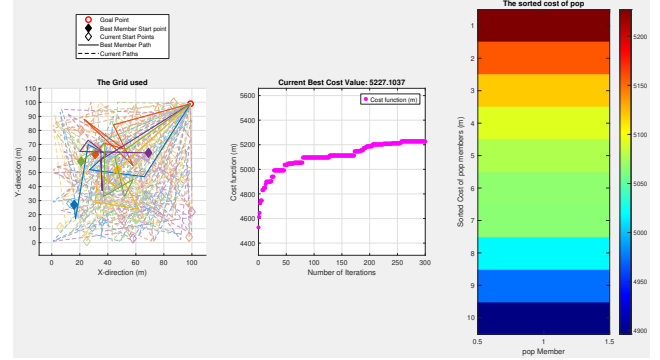


Fig. 5. APO algorithm results for the base case

APO, intriguing results prompted a closer look at the problem formulation, particularly the weights used. As shown in the cost function curve, the default APO configuration outperformed previous algorithms and, in some runs, matched or exceeded results achieved with double the AGVs or path points. However, as revealed in the left graph, the best protozoa showed compact trajectories for all AGVs, clustering near the right corner close to the goal. The higher cost function was, therefore, primarily due to high time minimization, as the longest exploration time taken by the AGVs was extremely short (e.g. going from an average of 44.6 hours for one protozoa in the first iteration to an average of 25.8 hours for the best protozoa in the last iteration). This was validated when testing with zero time weight, which resulted in more spacious trajectories covering most of the grid. Adjusting parameters like neighbor pairs and maximum proportion fraction had minimal impact on the cost function or computation time, though exploration slightly increased with the latter. Extending iterations to 500 showed the cost function still improving, with no signs of convergence. Increasing population size had little effect on the cost function but increased computation time by 1.4 times. Finally, as expected, doubling path points or AGVs improved fitness but at the cost of increased computation time and, for additional path points, higher battery requirements.

The convergence curve in Fig. 6 provides a comparative visualization of all algorithms, showcasing their performance under the same parameters specified in table I.

As can be seen in Fig. 6, APO continued to exploit the problem until the end, achieving the best solution by a significant margin. PSO stopped increasing around the 130th iteration with the previous 40 having a very close value. The GA was following closely, achieving a comparable solution

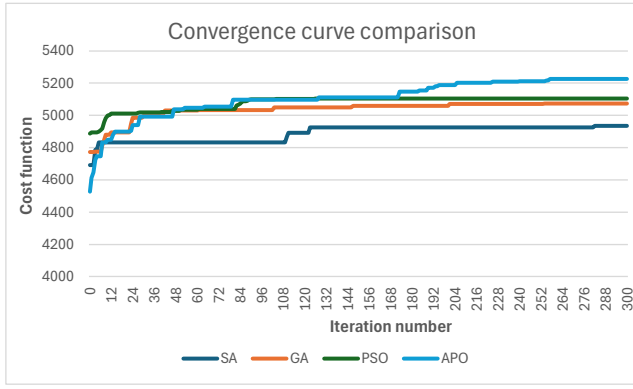


Fig. 6. Base case convergence curve for all algorithms

but in more iterations. In contrast, SA stagnated after the 5th iteration for a while, then exhibited a spike around the 110th iteration, and ultimately converged producing a significantly worse solution compared to the others.

Table II summarizes the comparison between all four algorithms. To ensure the precision of the results, each algorithm was executed 15 times under identical conditions by using the same PC. The specifications of the PC used for these runs are as follows: Intel Core i7-8850H CPU @ 2.60 GHz, 16 GB RAM, and an x64-based processor.

TABLE II
COMPARISON OF SA, GA, PSO, AND APO KEY PERFORMANCE
INDICATORS (KPI) OVER 15 RUNS

KPI	GA	SA	PSO	AP0
Average	5064.26045	4892.79834	5024.31844	5473.42
Best	5161.24760	4966.38130	5197.14690	6731.76
Standard Deviation	40.90384	45.81453	92.03482	466.06
Average elapsed Time	195.20862	33.59202	315.61849	221.79

APO emerged as the most effective, achieving the highest average elite value (5473.42) and best optimal solution (6731.76) among the 15 runs. However, it exhibited the highest deviation (466.06), indicating that the solutions may vary significantly across different runs and required moderate computational time (221.79 seconds). GA and PSO followed with balanced performance and lower deviation. GA was more repeatable having the lowest standard deviation (40.90) and slightly faster (195.21 seconds), while PSO provided better optimal solutions over all runs (5197.15) but required the longest runtime (315.62 seconds). SA was the fastest algorithm (33.59 seconds) but delivered the lowest average and best cost functions, showing its limitations in finding optimal solutions. In summary, APO is ideal for applications requiring optimal solutions, however, this comes at the expense of a high standard deviation of the results. GA and PSO offer a trade-off between solution optimality and repeatability. Specifically, PSO tends to achieve optimal individual solution across all runs compared to GA, whereas GA demonstrates a lower standard deviation, a better average solution across runs, and reduced computational complexity. SA is suitable for scenarios where computational complexity is crucial but

optimality can be compromised. The choice of algorithm ultimately depends on the problem's specific requirements for optimality, repeatability, and computational complexity.

IV. CONCLUSION

To conclude, this paper explored various optimization techniques for solving an arithmetic path-planning problem for multi-AGVs in search and rescue missions. The decision variables were the start points and paths, while the objective function aimed to maximize exploration coverage and minimize the total time required to reach the endpoints, subject to constraints such as battery life, minimum distance between consecutive points and avoiding collisions between AGVs. The algorithms evaluated included SA, a trajectory-based algorithm, and three population-based algorithms: GA, PSO, and APO. Among these, APO achieved the highest objective function values, primarily due to its exceptional ability to minimize time. However, this came at the cost of poor exploration, leaving large portions of the grid unexplored, a critical drawback for search and rescue applications. Adjusting the time weight improved area coverage. GA emerged as a strong alternative, offering a better balance between exploration maximization and time minimization. PSO followed, providing comparable performance, while SA, despite having the lowest objective function, delivered the fastest computation times. APO is recommended for applications where time minimization is prioritized. In cases where speed is the primary concern, SA emerges as the most effective choice.

REFERENCES

- [1] Z. Guo, Z. Wu, and X. Xu, "Transport route planning for agvs under fire hazard based on lstm and improved a* algorithm," *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, p. 09596518241280205, 2024.
- [2] E. Aarts and P. Van Laarhoven, "Simulated annealing: an introduction," *Statistica Neerlandica*, vol. 43, no. 1, pp. 31–52, 1989.
- [3] T. Turker, O. K. Sahingoz, and G. Yilmaz, "2d path planning for uavs in radar threatening environment using simulated annealing algorithm," in *2015 international conference on unmanned aircraft systems (ICUAS)*. IEEE, 2015, pp. 56–61.
- [4] S. Forrest, "Genetic algorithms: principles of natural selection applied to computation," *Science*, vol. 261, no. 5123, pp. 872–878, 1993.
- [5] X. Wu, Z. Feng, J. Zhu, and R. Allen, "Ga-based path planning for multiple auvs," *International Journal of Control*, vol. 80, no. 7, pp. 1180–1185, 2007.
- [6] D. Wang, D. Tan, and L. Liu, "Particle swarm optimization algorithm: an overview," *Soft computing*, vol. 22, no. 2, pp. 387–408, 2018.
- [7] S. L. Yu, Y. M. Bo, Z. M. Chen, and K. Zhu, "Vehicle path planning method based on particle swarm optimization algorithm," *Advanced Materials Research*, vol. 468, pp. 2745–2748, 2012.
- [8] X. Wang, V. Snášel, S. Mirjalili, J.-S. Pan, L. Kong, and H. A. Shehadeh, "Artificial protozoa optimizer (apo): A novel bio-inspired metaheuristic algorithm for engineering optimization," *Knowledge-Based Systems*, vol. 295, p. 111737, 2024.
- [9] Z. Guo, Y. Xia, J. Li, J. Liu, and K. Xu, "Hybrid optimization path planning method for agv based on kgwo," *Sensors*, vol. 24, no. 18, p. 5898, 2024.
- [10] R. Haba, M. Ohzeki, and K. Tanaka, "Travel time optimization on multi-agv routing by reverse annealing," *Scientific Reports*, vol. 12, no. 1, p. 17753, 2022.
- [11] T. Qiuyun, S. Hongyan, G. Hengwei, and W. Ping, "Improved particle swarm optimization algorithm for agv path planning," *IEEE Access*, vol. 9, pp. 33 522–33 531, 2021.